Tron Game Specification and Documentation

Yusupov Boburjon
Neptun: YTAJDI

December 6, 2025

1 Description of the Exercise

Create a game, with we can play the light-motorcycle battle (known from the Tron movie) in a top view. Two players play against each other with two motors, where each motor leaves a light trace behind of itself on the display. The motor goes in each seconds toward the direction, that the player has set recently. The first player can use the WASD keyboard buttons, while the second one can use the cursor buttons for steering. A player loses if its motor goes to the boundary of the game level, or it goes to the light trace of the other player. Ask the name of the players before the game starts, and let them choose the color their light traces. Increase the counter of the winner by one in the database at the end of the game. If the player does not exit in the database yet, then insert a record for him. Create a menu item, which displays a highscore table of the players for the 10 best scores. Also, create a menu item which restarts the game.

2 Analysis

The problem requires creating a graphical application that simulates a simplified version of the Tron light cycle game. The core requirements involve real-time movement, collision detection, and persistent data storage.

2.1 Key Components

- **Game Loop**: A mechanisms to update the game state at fixed intervals (timer).
- **Collision Detection**: Logic to determine if a player's position overlaps with walls, obstacles, or trails.
- **Input Handling**: distinct controls for two players sharing the same keyboard.
- **Data Persistence**: A database connection to store cumulative scores across sessions.
- **Level Generation**: An Algorithm to create distinct playable environments.

3 Implementation

3.1 Level Generation Algorithm

The level generation is handled by the `LevelGenerator` class. Instead of static files, we use a procedural approach to ensure infinite replayability.

Algorithm Strategy:

1. The number of obstacles is determined by the formula: $count = levelIndex \times 5$.
2. Random (x, y) coordinates are generated within the grid bounds.
3. **Safety Zones:** To ensure the game is playable, we define "safety zones" around the starting positions of Player 1 (top-left) and Player 2 (bottom-right).
4. If a generated coordinate falls inside a safety zone, it is discarded, and a new one is picked.
5. Valid coordinates are added to the list of obstacles.

3.2 Game Loop and Timing

The game uses a `javax.swing.Timer` set to trigger every 100ms. On each tick:

1. The model updates player positions based on their current direction.
2. Collision checks are performed.
3. The view (JPanel) is repainted to reflect the new state.

4 Events and Event Handlers

The application relies on the Swing Event Dispatch Thread (EDT) for handling user interactions and timer updates.

Event Source	**Event Type**	**Handler Action**
`Timer` (GamePanel)	`ActionEvent`	Calls `gameModel.update()` to advance game state and `repaint()` to update graphics.
`Keyboard` (GamePanel)	`KeyEvent`	Updates `Motor` direction: • WASD → Player 1 Direction • Arrows → Player 2 Direction • ESC → Return to Menu
`Start Button` (Menu)	`ActionEvent`	Validates names/colors and triggers `mainFrame.startGame()`.
`HighScores Button`	`ActionEvent`	Switches CardLayout to HighScore view and queries Database.

Table 1: Event Handling Mapping

5 Class Diagram

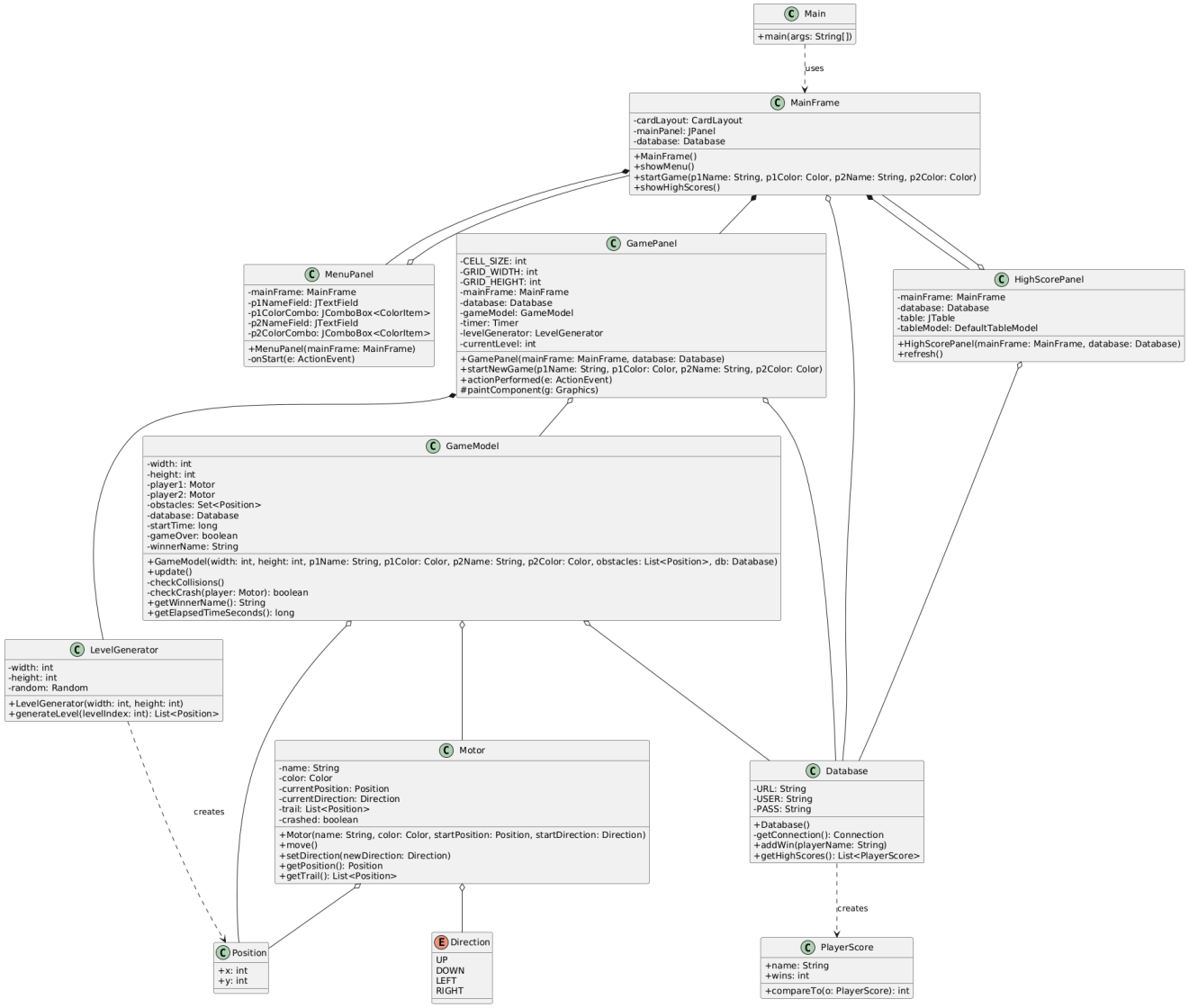

Figure 1: Class Diagram of the Tron Application

5.1 Class Overview

The application follows the Model-View-Controller (MVC) architectural pattern:

- **View (GUI)**: Classes in the ‘tron.gui‘ package (MainFrame, MenuPanel, GamePanel, HighScorePanel).
- **Model (Logic)**: Classes in the ‘tron.logic‘ package (GameModel, Motor, LevelGenerator, Position, Direction).
- **Data (DB)**: Classes in the ‘tron.db‘ package (Database, PlayerScore).
- **Controller**: Event handling is managed within the View classes (ActionListeners) delegating to the Model.

6 Class and Method Descriptions

6.1 Main Class (tron.Main)

Entry point for the application.

- `main(String[] args)`: Launches the application by creating the `MainFrame` on the Event Dispatch Thread.

6.2 Game Mechanics (tron.logic)

6.2.1 GameModel

Manages the core state of the game session.

- `update()`: Advances the game state by one tick. Moves players and checks for collisions.
- `checkCollisions()`: Determines if any player has crashed into walls, obstacles, or trails.
- `checkCrash(Motor player)`: detailed collision logic for a single player.
- `getWinnerName()`: Returns the name of the winner or "Draw".

6.2.2 Motor

Represents a player's entity.

- `move()`: Updates the motor's position based on its current direction and adds the new position to its trail.
- `setDirection(Direction newDirection)`: Changes the player's direction, preventing 180-degree turns.

6.2.3 LevelGenerator

Handles procedural generation of proper game levels.

- `generateLevel(int levelIndex)`: Creates a list of random `Positions` for obstacles. Ensures obstacles do not spawn in player starting safety zones.

6.3 Database Integration (tron.db)

6.3.1 Database

Handles JDBC connections to PostgreSQL.

- `addWin(String playerName)`: Increments the win count for a player in the ‘results‘ table. Uses an ‘INSERT ... ON CONFLICT DO UPDATE‘ statement.
- `getHighScores()`: Queries the top 10 players ordered by score.

6.4 User Interface (tron.gui)

6.4.1 MainFrame

The main container managing the application screens using ‘CardLayout‘.

- `showMenu()`: Switches to start screen.
- `startGame(...)`: Switches to the game view and initializes a new session.
- `showHighScores()`: Switches to the high score table.

6.4.2 GamePanel

Renders the game loop.

- `actionPerformed(ActionEvent e)`: The game loop tick handler.
- `paintComponent(Graphics g)`: Draws the grid, players, trails, and obstacles.

7 Test Cases

Unit tests are implemented using JUnit 4.

7.1 Logic Layer Tests

7.1.1 GameModelTest

- **testInitialization**: Verifies players start at correct positions.
- **testUpdateMovesPlayers**: checks that ‘update()‘ advances player coordinates.
- **testWallCollision**: Simulates a player hitting the wall and verifies game over state and winner designation.
- **testHeadOnCollision**: Simulates players crashing into each other, resulting in a draw.

7.1.2 MotorTest

- **testInitialization**: Checks initial state of a motor.
- **testMove**: Verifies movement logic and trail growth.
- **testDirectionChange**: checks valid 90-degree turns.
- **testInvalidDirectionChange**: checks that 180-degree turns are ignored.

7.1.3 LevelGeneratorTest

- **testGenerateLevelCount**: Verifies that the number of obstacles scales with level index.
- **testSafetyZones**: Ensures no obstacles are generated in the corners where players spawn.

7.1.4 DirectionTest

- **testEnumValues**: Basic verification of Direction enum constants.

7.1.5 PositionTest

- **testPositionValues**: checks record accessor methods.
- **testEqualsAndHashCode**: Verifies value-based equality for positions.

7.2 Test Summary

Test Class	Number of Tests
GameModelTest	4
MotorTest	4
LevelGeneratorTest	2
DirectionTest	1
PositionTest	2
Total	**13**

Table 2: Test Coverage Summary